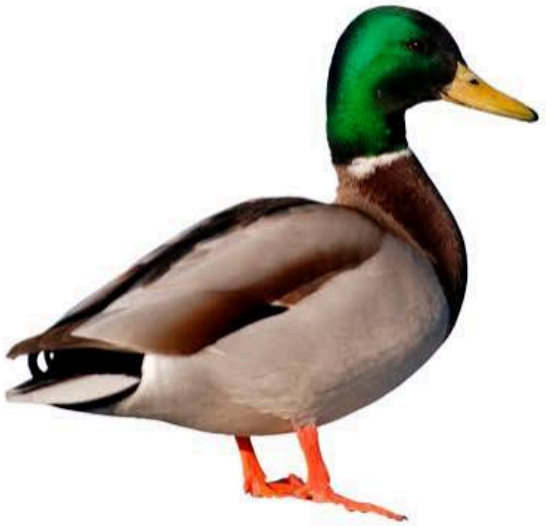
Ciência de Dados Aplicada ao Direito I

Aula 19

Requisições, APIs e raspagem



Prof. Bruno da Cunha de Oliveira - 2026/2

O que tem num repositório

Insper

README: o manual do projeto

O `README.md` é a primeira coisa que aparece na página de um repositório. Diz o que o projeto faz, como instalar e como rodar.

O que o grupo não entendeu, a IA ajuda a entender: colem o trecho num chat e peçam a explicação para alguém que nunca programou. Depois confiram no repositório se a explicação bate com o que está lá.

Na fase 2, o `README` é a porta de entrada do repositório que vocês entregam à banca.

Licença: o que se pode fazer com o código

Licença	O que permite, e o que pede em troca
MIT	usar, mudar, vender; manter o aviso e a licença
Apache 2.0	o mesmo, e dizer o que foi mudado
BSD 3-Clause	o mesmo, sem usar o nome dos autores para promover
GPL e AGPL	quem distribui versão mudada abre o código; na AGPL, também quem a oferece pela internet
proprietária	só quem o dono autorizar usa, mesmo com o código à vista
nenhuma	o mesmo que a proprietária

A licença do grupo: três caminhos

Caminho	Como fica
aberto	seguir com a licença da base, ou escolher uma no GitHub: **Add file**, **Create new file**, o nome `LICENSE` e o botão **Choose a license template**
fechado	licença proprietária: o repositório continua público para a banca, e só quem o grupo autorizar usa; o `LICENSE` diz "Copyright 2026 Grupo X. Todos os direitos reservados."
misto	uma parte aberta e outra fechada

Com uma base MIT, o grupo pode fechar o produto

O CÓDIGO DO GRUPO

a licença que o grupo escolher, inclusive a proprietária

O CÓDIGO QUE VEIO DA BASE

continua MIT, e o aviso de copyright da base fica no repositório

MIT, Apache 2.0 e BSD pedem só que o aviso e a licença da base fiquem junto do código que veio dela.

Com uma base GPL ou AGPL seria diferente: o produto inteiro teria de ficar aberto. Nenhuma das cinco bases é assim.

Misto: o PrairieLearn

O PrairieLearn, em que vocês fazem os notebooks, tem o código inteiro no GitHub, com três licenças:

Parte	Licença	O que permite
edição comunitária	AGPL	usar e mudar; quem oferece pela internet abre o código
edição Enterprise	proprietária	usar em produção só com contrato ou assinatura
código antigo, de Illinois	MIT	usar, mudar e vender, mantendo o aviso

Issue e pull request

Aba	O que tem nela
Issues	conversas sobre o projeto: uma dúvida, uma sugestão, um defeito encontrado. Não muda arquivo nenhum
Pull requests	propostas de mudança nos arquivos, que o dono lê, comenta e aceita ou recusa. A sigla é PR

AGENTS.md e CLAUDE.md: instruções para o assistente

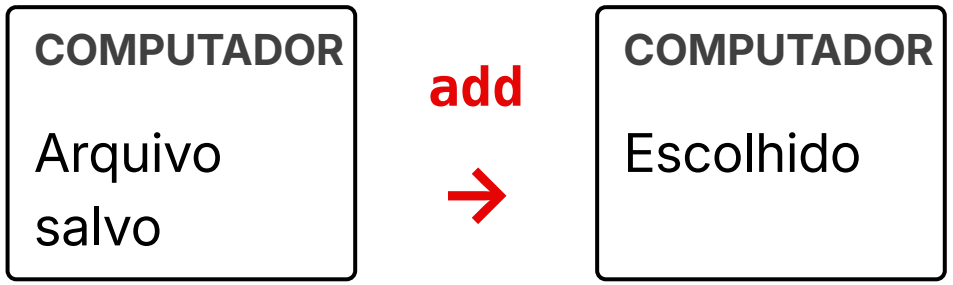
Arquivo	O que é
`AGENTS.md`	um formato aberto de instruções para assistentes de código; o Codex e outros leem
`CLAUDE.md`	as instruções que o Claude Code lê no começo de cada sessão; sem ele, o Claude Code lê o `AGENTS.md`

Na APS: escrevam ali o que o produto faz, como rodar e as regras do grupo, como "responda em português". O assistente de cada um de vocês começa sabendo, sem que ninguém precise repetir.

Registrar e enviar

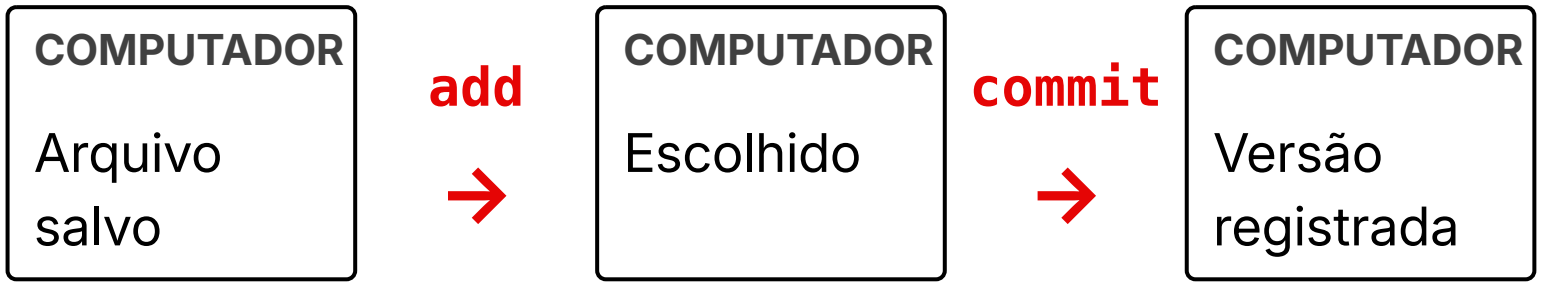
Insper

add, commit e push



`git add .` escolhe tudo o que mudou na pasta para entrar na próxima versão.

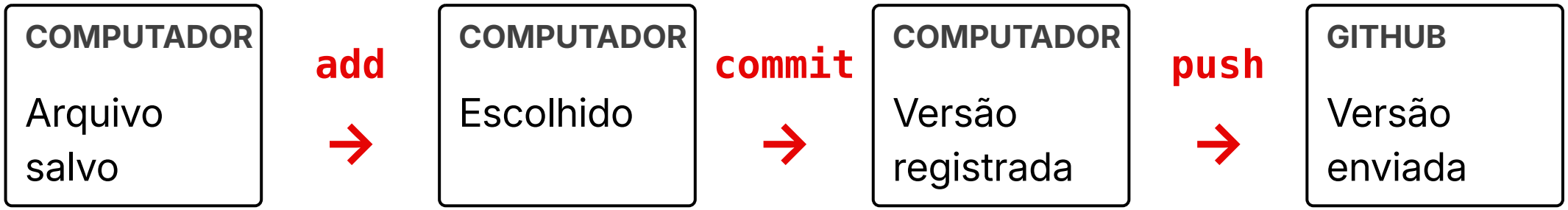
add, commit e push



`git add .` escolhe tudo o que mudou na pasta para entrar na próxima versão.

`git commit -m "..."` registra a versão no computador, com uma mensagem que diz o que mudou.

add, commit e push



`git add .` escolhe tudo o que mudou na pasta para entrar na próxima versão.

`git commit -m "..."` registra a versão no computador, com uma mensagem que diz o que mudou.

`git push` envia ao GitHub as versões registradas. Sem ele, ninguém do grupo nem da banca as vê.

Duas atividades no GitHub

Insper

Atividade 1: uma issue e um pull request

#	No repositório `bdcdo/praticas-github-cdad-2026-2`
1	Abrir uma issue explicando o que é uma issue, como você explicaria a um advogado que nunca programou
2	Fork, clone e, no VS Code, o arquivo `explicacoes/seu-usuario.md`, explicando o que é um pull request ao mesmo advogado
3	Registrar e enviar com `git add`, `git commit` e `git push`, e abrir o pull request a partir do fork

Até o início da aula de terça, 27/10, pelo formulário "issue e PR", no Blackboard. Escritos por você, sem assistente; o passo a passo está no guia.

Quanto vale a atividade 1

Quem	Ponto extra
Abriu a issue e o pull request	0,1
A melhor explicação de issue	mais 0,1
A melhor explicação de pull request	mais 0,1

O ponto extra vai para a prova final. A mesma pessoa pode ganhar os dois prêmios e chegar a 0,3. No empate, ganha quem enviou primeiro.

Atividade 2: o seu site pessoal

#	O site tem
1	Endereço `seu-usuario.github.io`, abrindo para qualquer pessoa
2	Uma apresentação sua
3	Link para o seu currículo Lattes
4	Link para o seu perfil no LinkedIn

Quanto vale a atividade 2

Link	Está atualizado quando
Lattes	a "Última atualização" é de 2026
LinkedIn	o perfil tem foto, título e a formação no Insper

Vale 0,1 ponto extra na prova final para quem tiver os quatro itens no site e os dois links atualizados até o início da aula de quarta, 25/11, e mandar o endereço por e-mail a `brunoco2@insper.edu.br`. O passo a passo está no guia.

O assistente de código

Insper

Passo 6 do guia: Codex ou Claude

Caminho	Onde roda	Que conta pede
Codex, aplicativo	programa separado	ChatGPT, até a gratuita
Codex, extensão	dentro do VS Code	ChatGPT Edu, Plus ou maior
Claude, aplicativo	à parte, aba **Code**	Claude Pro ou Max
Claude Code, extensão	dentro do VS Code	Claude Pro ou Max

Com o ChatGPT Edu do Insper, o Codex funciona no aplicativo e na extensão. Escolham um, instalem e abram nele a pasta `pratica-git`.

Se o assistente não registrar ou não enviar

Se acontecer	O que fazer
o assistente pede permissão para registrar ou enviar	aprovar
o envio feito pelo assistente falha	rodar `git push` no terminal do VS Code

Um pedido ao assistente

Façam este pedido ao assistente:

> Acrescente ao `prazos.py` a função `dias_restantes(prazo, decorridos)`, que devolve quantos dias faltam. Depois mostre quantos dias restam de um prazo de 15 dias em dobro, passados 12 dias. Registre a versão e envie ao GitHub.

Antes de rodar o programa: que número novo tem que aparecer?

A resposta antes do código

RESPOSTA

```
30
60
18
```

As duas primeiras linhas já estavam no arquivo. A nova é 18: o dobro de 15 é 30, e 30 menos 12 é 18.

Vocês sabem a resposta antes de o assistente escrever uma linha, e é isso que permite conferir o que ele entregou.

Pedir ao assistente que desfaça

A função nova não serve mais. Façam este pedido:

> Desfaça o último commit com um commit novo, sem apagar o histórico, e envie ao GitHub.

Quantas versões o histórico do seu fork passa a ter? A função `dias_restantes` some do histórico?

Desfazer acrescenta uma versão

RESPOSTA

Cinco: as duas que vieram do repositório do curso, a de vocês, a do assistente e a que desfaz.

A função continua no histórico, na quarta versão. Desfazer não apaga nada: registra uma versão nova com o conteúdo de antes.

Entreguem o endereço do seu fork pelo formulário "fork da aula 18", no Blackboard.

Um pedido a um site

Um tribunal para fazer pedidos

Um site do curso que imita um tribunal: tudo nele é inventado, e as páginas e os dados têm o formato dos de um tribunal de verdade. Abram no navegador:

```
tribunal-ficticio-cdad.fly.dev/cjpg/
pesquisar?conteudoPesquisa=dano+moral
```

O que vem depois do ? são os parâmetros do pedido: aqui, o que se pesquisa.

O mesmo endereço, pelo Python: o que aparece na tela?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/cjpg/pesquisar"
pesquisa = {"conteudoPesquisa": "dano moral"}
resposta = requests.get(url, params=pesquisa)
print(resposta.status_code)
print(resposta.text[:60])
```

O navegador desenha, o Python recebe o texto

RESPOSTA

```
200
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="ut
```

A página que o navegador desenhou chega ao Python como texto, em HTML: as etiquetas entre < e > dizem ao navegador como mostrar cada pedaço. O 200 é o código que o servidor mandou junto, e quer dizer que deu certo.

Um pedido e uma resposta

Animação no navegador: `animacao-requisicao.html`, cenário `pedido`.

O que vai e o que volta

O pedido leva	A resposta traz
o método: GET, pedir para ler	o código: 200, 404, ...
o endereço e os parâmetros	os cabeçalhos: o tipo do conteúdo, o tamanho
os cabeçalhos: quem pede, o que aceita	o corpo: o HTML, ou os dados

Nem todo pedido dá certo

Um processo pelo número: o que aparece na tela?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
numero = "0000101-95.2026.8.99.0001"
resposta = requests.get(url, params={"numero": numero})
print(resposta.status_code)
dados = resposta.json()
if resposta.status_code == 200:
    print(dados["classe"]["nome"])
else:
    print(dados["erro"])
```

O pedido que acha, passo a passo

Animação no navegador: `animacao-http.html`, cenário `achou`.

Pedido, resposta, atributo e método

RESPOSTA

```
200
Procedimento Comum Cível
```

`requests.get` devolve um objeto: a resposta. `resposta.status_code` é um atributo, o código que o servidor mandou. `resposta.json()` é um método: transforma o texto da resposta num dicionário.

Um número fora do cadastro: o que aparece na tela?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
numero = "0000999-11.2026.8.99.0001"
resposta = requests.get(url, params={"numero": numero})
print(resposta.status_code)
dados = resposta.json()
if resposta.status_code == 200:
    print(dados["classe"]["nome"])
else:
    print(dados["erro"])
```

O pedido que não acha, passo a passo

Animação no navegador: `animacao-http.html`, cenário `nao-achou`.

O 404 também é resposta

RESPOSTA

```
404
processo não encontrado
```

O servidor respondeu, com o código 404, e o corpo explica o motivo. Sem o `if`, o programa pediria `dados["classe"]` a um dicionário que não tem essa chave e pararia em `KeyError`.

Um processo que existe, mas: o que aparece na tela?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
numero = "0002828-27.2026.8.99.0001"
resposta = requests.get(url, params={"numero": numero})
print(resposta.status_code)
dados = resposta.json()
if resposta.status_code == 200:
    print(dados["classe"]["nome"])
else:
    print(dados["erro"])
```

O 403: o processo existe, e o acesso é negado

RESPOSTA

```
403
processo em segredo de justiça
```

O servidor sabe que o processo existe e se recusa a mostrá-lo a quem consulta sem ser parte nem advogado. O 404 dizia "não achei"; o 403 diz "achei, e você não pode ver".

A área do advogado: o que aparece na tela?

```
import requests
site = "https://tribunal-ficticio-cdad.fly.dev"
resposta = requests.get(site + "/api/advogado/processos")
print(resposta.status_code)
print(resposta.json()["erro"])
```

O 401: quem está pedindo?

RESPOSTA

```
401
identificação necessária
```

A rota só atende quem se identifica, com login ou chave, e o pedido não levou nenhuma. O 401 pergunta quem pede; o 403 responde que quem pediu não pode ver.

Cada pedido, um código

Animação no navegador: `animacao-requisicao.html`, cenário `codigos`.

O primeiro dígito diz a família

Código	Família	No Tribunal Fictício
200	2: atendido	achou o processo
404	4: não atendido	o número não está no cadastro
401	4: não atendido	a rota exige identificação, e ninguém se identificou
403	4: não atendido	na consulta pública, o acesso é negado: segredo de justiça
429	4: não atendido	pedidos demais em pouco tempo
500	5: falha do servidor	o servidor quebrou ao responder; nenhum pedido daqui o produz

Por que um tribunal responde 403

Código de Processo Civil, art. 189, caput (trecho) e § 1º

"Os atos processuais são públicos, todavia tramitam em segredo de justiça os processos: [...] II - que versem sobre casamento, separação de corpos, divórcio, separação, união estável, filiação, alimentos e guarda de crianças e adolescentes; [...]"

"§ 1º O direito de consultar os autos de processo que tramite em segredo de justiça e de pedir certidões de seus atos é restrito às partes e aos seus procuradores."

A consulta pública não identifica ninguém e não mostra esses autos a quem pedir: responde 403. As partes e os advogados veem os autos por outro caminho, identificados.

Raspar ou pedir os dados

Insper

Onde está o número na página

Na consulta do Tribunal Fictício, aberta no começo da aula, botão direito sobre o número do primeiro processo, e "Inspecionar". O número está dentro de uma etiqueta:

```
<span class="fonteNegrito">0000101-95.2026.8.99.0001</span>
```

Raspar é escrever o programa que acha essa etiqueta no texto que chegou e tira dela o que interessa. Se o site mudar o nome da etiqueta, o programa deixa de achar.

O mesmo processo, pela página

Animação no navegador: `animacao-raspar-ou-api.html`, cenário `html`.

O mesmo processo, pela API

Animação no navegador: `animacao-raspar-ou-api.html`, cenário `json`.

Página ou API

	Raspar a página	Usar a API
Feita para	quem lê na tela	outro programa
Chega como	HTML, com o desenho junto	JSON, só os dados, com nome
Para tirar o dado	achar a etiqueta certa	ler a chave do dicionário
Se o site mudar o desenho	o programa quebra	costuma continuar igual

API é a porta que o site abre para programas. Quando existe, é o primeiro lugar onde procurar.

Uma API de verdade: a da Câmara dos Deputados

A Câmara publica os dados dela numa API aberta, sem cadastro:

```
dadosabertos.camara.leg.br/api/v2/proposicoes
```

Abram o endereço no navegador: em vez de página, vem JSON.
A documentação, com cada parâmetro que a API aceita, está em `dadosabertos.camara.leg.br/swagger/api.html`.

Projetos de lei sobre inteligência artificial em 2026: rodem

```
import requests
url = "https://dadosabertos.camara.leg.br/api/v2/proposicoes"
pesquisa = {"keywords": "inteligência artificial",
            "siglaTipo": "PL", "ano": 2026, "itens": 3}
resposta = requests.get(url, params=pesquisa)
print(resposta.status_code)
for pl in resposta.json()["dados"]:
    print(pl["siglaTipo"], pl["numero"], pl["ano"])
```

Três projetos, e o resto em outras páginas

RESPOSTA

```
200
PL 19 2026
PL 83 2026
PL 212 2026
```

Resultado de 14/10/2026: a lista cresce quando um projeto novo é apresentado. `"itens": 3` pediu três por página. A própria resposta traz os endereços da página seguinte e da última, em `resposta.json()["links"]`: a última era a 34.

A busca do Tribunal Fictício, pela API

A consulta tem uma API ao lado, `/api/processos`, no formato da API pública do CNJ, o Datajud. A página 1 da busca "dano moral" volta assim:

```
{"hits": {"total": {"value": 24, ...},
          "hits": [{"_source": {"numeroProcesso": ...}},
                   ...]}}
```

24 processos no total, dez por página: cada página é um pedido, com `pagina` nos parâmetros.

Três páginas da busca: quantos processos?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processos"
resultados = []
for pagina in range(1, 4):
    pesquisa = {"pesquisa": "dano moral", "pagina": pagina}
    resposta = requests.get(url, params=pesquisa)
    dados = resposta.json()
    for item in dados["hits"]["hits"]:
        resultados.append(item["_source"]["numeroProcesso"])
print(len(resultados))
```

Página por página

Animação no navegador: `animacao-paginacao.html`, cenário `tres`.

Vinte e quatro, em três pedidos

RESPOSTA

```
24
```

10 na primeira página, 10 na segunda e 4 na terceira. Cada página é um pedido; o `for` de fora pede, e o de dentro junta. É o que o `paginas=` do juscraper faz por dentro.

Uma página a mais

Animação no navegador: `animacao-paginacao.html`, cenário `uma-a-mais`.

Avaliação da disciplina, agora

Dez minutos para a avaliação discente do Insper, antes do intervalo.

PROVISÓRIO: link e instruções a definir.

O que o navegador vê e o Python não

Inspecionar uma página de jornal

Aba	O que mostra
Elementos	o HTML da página, etiqueta por etiqueta
Rede	cada pedido que a página fez ao abrir: o HTML, as imagens, e os dados que chegaram depois

O mesmo "Inspecionar" do número do processo, agora na página de um jornal. A aba Elementos é a que vocês já usaram; na aba Rede, recarreguem a página e contem os pedidos.

Três linguagens numa página

Linguagem	O que faz
HTML	o conteúdo, em etiquetas
CSS	a aparência: cor, fonte, posição
JavaScript	o programa que o navegador roda depois que a página chega: pede mais dados, preenche a lista, responde ao clique

É a linguagem que roda no navegador, e a dos sites que os grupos vão construir.

Um painel de processos

Abram no navegador:

```
tribunal-ficticio-cdad.fly.dev/painel?pesquisa=dano+moral
```

A lista mostra "Carregando..." por um instante, e depois os processos. Agora "Inspecionar", aba Rede: recarreguem a página e procurem, entre os pedidos, um a `/api/processos`.

O script que vem com a página

```
const url = "/api/processos?pesquisa=dano+moral";
const resposta = await fetch(url);
const dados = await resposta.json();
const lista = document.getElementById("lista");
lista.innerHTML = "";
for (const item of dados.hits.hits) {
  const linha = document.createElement("li");
  linha.textContent = item._source.numeroProcesso;
  lista.append(linha);
}
```

O navegador roda o script

Animação no navegador: `animacao-pagina-javascript.html`, cenário `navegador`.

A mesma página, pelo Python: o que aparece na tela?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/painel"
pesquisa = {"pesquisa": "dano moral"}
resposta = requests.get(url, params=pesquisa)
print(resposta.status_code)
print("Carregando" in resposta.text)
print("0000101" in resposta.text)
```

O requests não roda script

Animação no navegador: `animacao-pagina-javascript.html`, cenário `requests`.

O requests recebe a página antes do script

RESPOSTA

```
200
True
False
```

O `requests` traz o texto da página e não roda o programa que veio junto: a lista fica em "Carregando...". Os processos chegam num segundo pedido, que o script faz a `/api/processos`. Esse é o pedido que o programa em Python deve fazer.

Como o site lembra de vocês

Insper

Pedidos demais, mas de quem?

A paginação pediu três páginas seguidas, e um raspador de verdade pede centenas. Na tabela dos códigos, o 429 era "pedidos demais em pouco tempo".

Só que o servidor responde cada pedido como se fosse o primeiro, sem lembrar do anterior. Como ele sabe que vários pedidos vêm do mesmo programa?

Dois pedidos na mesma sessão: rodem e comparem com o vizinho

```python
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
numero = "0000101-95.2026.8.99.0001"
sessao = requests.Session()
primeira = sessao.get(url, params={"numero": numero})
print(primeira.headers["Set-Cookie"])
print(sessao.cookies["sessao"])
segunda = sessao.get(url, params={"numero": numero})
print(segunda.status_code)
print("Set-Cookie" in segunda.headers)
```

O cookie vai e volta

Animação no navegador: `animacao-cookie.html`, cenário `sessao`.

O cookie é o bilhete que o site devolve

RESPOSTA

```
sessao=297ba087; Path=/; HttpOnly
297ba087
200
False
```

Na primeira resposta, o servidor manda um cookie: um código que a `requests.Session()` guarda e leva de volta em cada pedido seguinte. O servidor reconhece a sessão e não manda outro. O código de cada um saiu diferente: o site distingue uma sessão da outra, sem saber quem é a pessoa.

Dez pedidos seguidos: o que aparece na tela?

```
import requests
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
numero = "0000101-95.2026.8.99.0001"
sessao = requests.Session()
codigos = []
for vez in range(10):
    resposta = sessao.get(url, params={"numero": numero})
    codigos.append(resposta.status_code)
print(codigos)
print(resposta.headers["Retry-After"])
```

O 429: pedidos demais

RESPOSTA

```
[200, 200, 200, 200, 200, 429, 429, 429, 429, 429]
2
```

O servidor conta os pedidos de cada cookie: mais de cinco em dois segundos, e ele para de atender. O cabeçalho `Retry-After` diz quantos segundos esperar.

E com meio segundo entre os pedidos?

Na célula seguinte, com a mesma `sessao`:

```
import time
codigos = []
for vez in range(10):
    resposta = sessao.get(url, params={"numero": numero})
    codigos.append(resposta.status_code)
    time.sleep(0.5)
print(codigos)
```

Dez pedidos, com e sem espera

Animação no navegador: `animacao-cookie.html`, cenário `limite`.

Esperar também faz parte do programa

RESPOSTA

```
[200, 200, 200, 200, 200, 200, 200, 200, 200, 200]
```

`time.sleep(0.5)` para o programa por meio segundo. Com a pausa, cabem quatro pedidos em cada dois segundos, e o servidor atende todos. Abrir uma sessão nova para escapar do limite só esconde quem pede, e um tribunal de verdade conta também pelo endereço de internet (IP) de onde os pedidos saem. As bibliotecas de coleta esperam entre um pedido e outro.

Bibliotecas que já fazem isso

De onde veio o juscraper

O `pip install juscraper` baixou o pacote do PyPI, o catálogo público de bibliotecas de Python: `pypi.org/project/juscraper`. A página diz a versão, 0.4.0, e do que ela precisa:

```
pandas<3.0.0,>=2.0.0
requests>=2.32.3
```

Por isso o `requests` já estava no computador de vocês: veio junto com o juscraper.

Nem toda biblioteca está no PyPI

O raspe coleta de fontes oficiais e da imprensa: Presidência, Câmara, Senado, IPEA, CAPES e Folha, entre outras. Ele não está no PyPI, e se instala direto do GitHub:

```
pip install git+https://github.com/bdcdo/raspe.git
```

```
import raspe
camara = raspe.camara()
dados = camara.raspar(pesquisa="educação")
```

O cjpg por dentro: qual linha pede ao tribunal?

juscraper 0.4.0, `cjpg_download.py`. Trecho, com cortes.

```
query = {
    'dadosConsulta.pesquisaLivre': pesquisa,
    ...
    'dadosConsulta.ordenacao': 'DESC'
}

return session.get(f"{u_base}cjpg/pesquisar.do", params=query)
```

E onde entrou o "golpe do pix" de `tjsp.cjpg("golpe do pix")`?

O mesmo get, numa sessão

RESPOSTA

A última linha. `session.get(...)` é o `get` do `requests`, chamado numa sessão que o juscraper cria uma vez e reusa: é a sessão que guarda o cookie do tribunal entre um pedido e outro. `params=query` leva a pesquisa, e o "golpe do pix" entrou em `'dadosConsulta.pesquisaLivre': pesquisa`.

O resto do `cjpg` raspa a página que voltou: acha as etiquetas, tira os campos e monta a tabela.

Uma consulta sem self: o que aparece na tela?

```
url = "https://tribunal-ficticio-cdad.fly.dev/api/processo"
class Consulta:
    def __init__(self, url):
        self.url = url
    def endereco(numero):
        return self.url + "?numero=" + numero
consulta = Consulta(url)
print(consulta.endereco("0000101-95.2026.8.99.0001"))
```

O objeto antes do ponto é o self

RESPOSTA

```
TypeError: Consulta.endereco() takes 1 positional
argument but 2 were given
```

O programa passou um argumento, e o Python diz que chegaram dois. Em `consulta.endereco(...)`, ele entrega o objeto antes do ponto como primeiro argumento do método. Com `def endereco(self, numero):`, o programa mostra o endereço do pedido, com o número depois do `?`.

O raspador do Acre: tjac.cjsg funciona?

juscraper 0.4.0, `tjac/client.py`. A classe inteira, sem a linha de documentação.

```
class TJACScraper(EsajSearchScraper):
    BASE_URL = "https://esaj.tjac.jus.br/"
    TRIBUNAL_NAME = "TJAC"
```

É a classe do objeto que `tjac = jus.scraper("tjac")` cria. Ela não escreve nenhum `cjsg`.

As classes do juscraper

Animação no navegador: `animacao-heranca.html`, cenário `arvore`.

Onde o Python acha o método

Animação no navegador: `animacao-heranca.html`, cenário `procura`.

Funciona: o método vem da classe de cima

RESPOSTA

`class TJACScraper(EsajSearchScraper)` diz que o raspador do Acre herda tudo o que o `EsajSearchScraper` tem. Entre os tribunais do juscraper, seis usam o e-SAJ (TJSP, TJAC, TJAL, TJAM, TJCE e TJMS), e o código que pede e lê as páginas do e-SAJ foi escrito uma vez, na classe de cima. Cada tribunal só diz o próprio endereço.

O `TJSPScraper` também herda, e escreve de novo o `cjsg`, porque o TJSP tem diferenças: o método dele vale no lugar do herdado.

Assistentes que constroem raspadores

Insper

Onde o assistente erra

O modelo aprendeu com o código que existia quando foi treinado. Biblioteca pequena ou recente, como o juscraper, aparece pouco, e o assistente completa com o nome que parece certo:

```
tjsp.buscar_decisoes("golpe do pix")
```

```
AttributeError: 'TJSPScraper' object
has no attribute 'buscar_decisoes'
```

Uma skill: o manual que o assistente lê

Uma skill é um texto com instruções, que o assistente lê quando a tarefa pede: quais tribunais, quais métodos, quais parâmetros. No Claude Code:

```
/plugin marketplace add lab-dados/labdados-skills
/plugin install juscraper@labdados-skills
```

Depois disso, peçam ao assistente uma coleta com o juscraper e comparem com o que ele fazia antes.

E quando ninguém escreveu o raspador do site?

Skill	Constrói
`juscraper-builder`	o raspador de um tribunal que o juscraper ainda não cobre
`raspe-builder`	o raspador de uma fonte oficial ou de imprensa nova para o raspe
`raspador-builder`	um programa avulso que coleta qualquer site e salva em CSV

```
/plugin install juscraper-builder@labdados-skills
/plugin install raspe-builder@labdados-skills
/plugin install raspador-builder@labdados-skills
```

O que a skill de construir faz

#	Passo
1	abre o site, num navegador que o assistente controla ou só com `requests`, e olha os pedidos, como a aba Rede
2	procura a API por baixo da página antes de raspar o HTML
3	escreve o raspador só com `requests`, sem navegador
4	testa, com um segundo entre pedidos e respeitando o `robots.txt`, o arquivo em que o site diz quais robôs aceita

O navegador é o Playwright MCP, e MCP é o jeito de ligar uma ferramenta ao assistente. Só a `raspe-builder` pede para instalá-lo:

```
claude mcp add playwright -- npx @playwright/mcp@latest
```

Peçam ao assistente o raspador da busca do Tribunal Fictício

O pedido: um programa que colete número e classe de todos os processos da busca "dano moral" no Tribunal Fictício, em todas as páginas, e salve em CSV.

Enquanto ele trabalha: ele raspou a página ou achou a API? Quantos processos vieram?

O que conferir no raspador que chegou

RESPOSTA

A própria consulta informa o total: "Resultados 1 a 10 de 24". Se o CSV tiver 10 linhas, o raspador parou na primeira página; se a coluna da classe vier vazia, ele não leu o campo. Se raspou o HTML, funciona hoje e pode quebrar quando o site mudar o desenho; a API existe, em `/api/processos`.

Ninguém leu o código: o total que o site informa, a coluna conferida e a pergunta sobre o caminho bastaram.

Poder pedir não é poder coletar

O site diz quais robôs aceita

Todo site pode publicar um `robots.txt` dizendo o que cada robô pode visitar. O da Folha, em 14/10/2026, nomeava 177 robôs. Trecho:

```
User-agent: ClaudeBot
Disallow: /

User-agent: GPTBot
Allow: /
```

O arquivo é um pedido, e não uma trava: nada impede tecnicamente um programa de ignorá-lo.

Dado público ainda é dado pessoal

Lei Geral de Proteção de Dados, art. 7º, § 3º
"O tratamento de dados pessoais cujo acesso é público deve considerar a finalidade, a boa-fé e o interesse público que justificaram sua disponibilização."

Uma sentença publicada traz nome, às vezes CPF, doença, salário.

Quais destas coletas vocês fariam, e com que cuidado?

#	Coleta
1	Todas as sentenças de um tema no TJSP, com um pedido atrás do outro, sem intervalo
2	As notícias de um jornal com assinatura paga, para montar uma base pública
3	Os nomes e os diagnósticos das partes em ações de saúde, para uma pesquisa
4	Um site que bloqueia robôs, usando um navegador que finge ser uma pessoa

Uma raspagem de vocês

Insper

Atividade: uma raspagem com o juscraper

#	No seu clone de `praticas-github-cdad-2026-2`
1	Em casa, uma coleta com o juscraper sobre um tema de vocês, salva em CSV; na rede do Insper, 55 pessoas saem pelo mesmo endereço de internet (IP), e o tribunal pode bloquear todas de uma vez
2	O código e o CSV na pasta `raspagens/seu-usuario/`
3	`git add`, `git commit` e `git push`, e o pull request aberto a partir do fork

Vale 0,1 ponto extra na prova final, com o pull request aberto até o início da aula de terça, 27/10.

Na próxima aula

Como um programa conversa com um modelo de linguagem, e o que acontece entre a pergunta e a resposta.

Até o início dela, terça, 27/10: o pull request da raspagem, e a issue e o pull request da atividade 1.